

Operating system principles 4

Paul Blondel

CNAM

September 1st, 2018

- 1 Recall
- 2 Introduction
- 3 Allocation approaches
- 4 Free space management
- 5 Repositories
- 6 Partitions
- 7 File system operations
- 8 Protections

Recall:

- The *main memory* is volatile: computer **turned off** = **no more data**
- **All programs and data** are stored in the *mass storage memory*

But how is ensured the conservation / organization of the data in the *mass storage memory*?

How?

This is the goal of the **File System** (or FS)

- 1 Recall
- 2 Introduction**
- 3 Allocation approaches
- 4 Free space management
- 5 Repositories
- 6 Partitions
- 7 File system operations
- 8 Protections

A "file" has **two representations** in the system:

- The "**Logical File**" representation: file representation in the *main memory* (what the user can see about the data)
- The "**Physical File**" representation: file representation in the *mass storage memory* (data storage)

What does the FS?

The **FS** handles these two levels of file representation and ensures the matching between the two.

The "*Logical File*" representation is either:

- a **data structure** (as in a programming language) with *creation*, *opening*, *closing* and *destruction* operations:
 - ▶ **creating and opening a logical file: link created** with the *mass storage memory*
 - ▶ **closing and destructing a logical file: cut this link** with the *mass storage memory*
- or recordings of data (that can be read from the physical file) with **three different access modes**:
 - ▶ **sequential mode, indexed mode** and **direct mode**

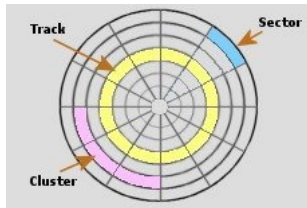
The three different modes:

- **Sequential access:** recordings are processed in their order
 - ▶ Here a read operation **gives the content at the current position**
 - ▶ The **read operation also moves to the next position**¹
 - ▶ A file is accessed either in read only mode or in write mode only
- **Indexed access:** recordings can be accessed directly giving a key
 - ▶ The position given is an *access key*
 - ▶ For **each recording: an access key is associated**
 - ▶ This access key is **used to retrieve the right recording**
- **Direct access:** Recordings can be accessed directly giving a position
 - ▶ The position is relative to the beginning of the file

¹This access mode naturally follows HDD physical properties (circular readings and writings on magnetic disks)

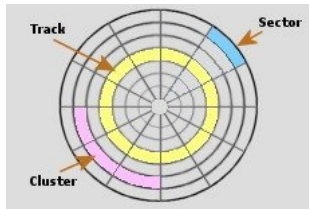
The "*Physical file*" representation is:

- The physical representation as stored in *mass storage memory*
- Most *mass storage memories* nowadays are still made of disks (HDD):



The "*Physical file*" representation is:

- The physical representation as stored in *mass storage memory*
- Most *mass storage memories* nowadays are still made of disks (HDD):



Optimization

Read/write accesses: sectors are **grouped in clusters** (or blocks)

When storing a file into the *mass storage memory*:

- The file is **stored into *blocks*** (groups of sectors)
- It becomes a *Physical File*: a set of allocated physical *blocks*

Four different approaches to allocate *blocks* in *mass storage memory*:

- Contiguous allocation
- Allocation per area
- Allocation of linked-blocks
- Indexed allocation

- 1 Recall
- 2 Introduction
- 3 Allocation approaches**
- 4 Free space management
- 5 Repositories
- 6 Partitions
- 7 File system operations
- 8 Protections

Contiguous?

The file **must** occupy a set of contiguous physical blocks

- Advantages:
 - ▶ **Simple** approach
 - ▶ **Good performances** ("few" physical movements of the reading head)
- Drawbacks:
 - ▶ Need to know in advance the final file size
 - ▶ The memory is **not optimizely used**²
 - ▶ The **fragmentation of the memory is a big problem** here
 - ▶ If the **size of the file increases: big problem**³

²First area, best area and worst area strategies can also be used here

³If the size of the file cannot be increased: an error occurs

Per area?

Here file *blocks* are allocated in two areas:

- a primary area (containing the main blocks)
- a secondary area (useful if the file size increase)

- **Advantages:**

- ▶ **More flexibility** to allocate blocks if the file size increases

- **Drawbacks:**

- ▶ **More physical movements** of the reading head (slow down a little)
- ▶ We cannot extend too much the secondary area (**not that flexible!**)

Linked-blocks?

File blocks are **linked together** (each block contains address of next block)⁴

- **Advantages:**

- ▶ **Extending a file is easy**, we just: add a new block and link it
- ▶ The **fragmentation of the memory**: is not a problem any longer!
- ▶ **Don't need to know the final memory size** of the file

- **Drawbacks:**

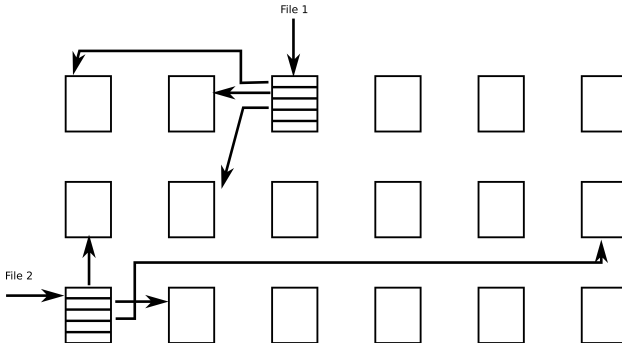
- ▶ Can **only be used in sequential access mode**
- ▶ **Addresses take place** in the blocks (waste of space!)

⁵A variant to this approach is the FAT (File Allocation Table) approach: here a table keeps the next block positions of each file (the last next block position being identifiable)

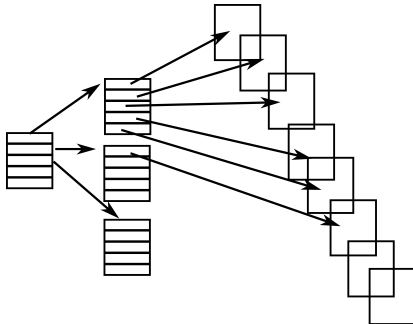
Indexed allocation

It removes two drawbacks of the linked-blocks allocation approach!

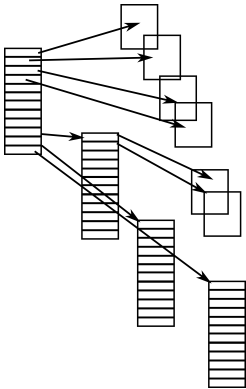
All block addresses are stored in one specialized block (named *index*):



If not enough space in one block to store all the addresses, we use **multilevel indexing**:



On *UNIX* a modified *indexed allocation* approach is used:



- First block is an index with 13 entries
- 10 first entries: addresses to blocks
- 3 last blocks: redirect to other index blocks

- 1 Recall
- 2 Introduction
- 3 Allocation approaches
- 4 Free space management**
- 5 Repositories
- 6 Partitions
- 7 File system operations
- 8 Protections

Free space management?

Managing free *blocks* is necessary to find them later and allocate memory

- When creating a new file: the FS looks for free blocks in a list
- When a file is destructed: new free blocks are "memorized" in a list

There are two ways to identify the list of free blocks⁵:

- using a **vector of bits**
- by **linking the free blocks together**

⁵With *FAT*: there is already this information in another column of the table of blocks

- **Vector bit management:**

- ▶ A bit set to **0 means: the block is free**
- ▶ A bit set to **1 means: the block is not free**
- ▶ Example: 1 1 0 0 1 1 0 1 1 0 1 (1st and 2nd block not free, etc)
- ▶ Vector size = number of blocks

- **Linked free blocks management:**

- ▶ Finding N free blocks = traverse the linked list
- ▶ Traversing the linked list **step after step: often slow**
- ▶ Variant: in free blocks are stored the number of following free blocks
- ▶ Jumping M consecutive free blocks = faster traversal

- 1 Recall
- 2 Introduction
- 3 Allocation approaches
- 4 Free space management
- 5 Repositories**
- 6 Partitions
- 7 File system operations
- 8 Protections

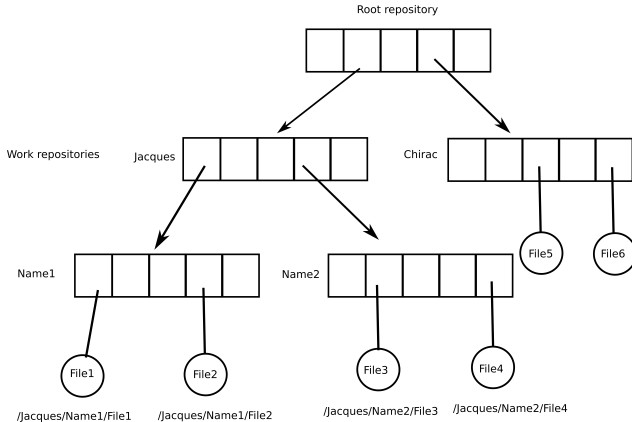
Repository?

Repositories contain *FS* information (for some concerned files)

The information for each file are:

- Name of Logical File
- Physical address in the mass storage memory (to access the blocks)
- The file type (.doc, .exe, .o, .gif, etc.)
- The size of the file (in ko or in block numbers)
- The name of the file owner
- Access rights (reading, writing and executing rights)

In most *FS*, repositories can have a N-levels structure, here with 2-levels:

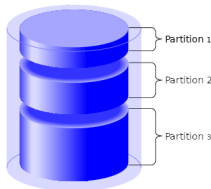


- 1 Recall
- 2 Introduction
- 3 Allocation approaches
- 4 Free space management
- 5 Repositories
- 6 Partitions**
- 7 File system operations
- 8 Protections

Partitions?

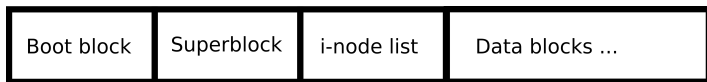
The *FS* may contain several thousands of files, in that case:

- More convenient to **split the system in partitions**
- Each partition **manages its part of the files**
- It simplifies the management of the data



- To access the content of a partition: partition linked to a repository (mounting)
- When the partition is not needed any longer: link broken with the repository (unmounting)

On *UNIX*, a partition is divided as follows:



- **Boot block:** occupy first sector and contains code to initialize the *FS*
- **Superblock:** contains information about the *FS* of the partition (*i-node* list, addresses, etc)
- **i-node list:** list of *i-nodes* (one *i-node* for each created file in the *FS*)

A file is physically represented by a descriptor called "*i-node*"

An *i-node* contains: the id of the file owner, the file type and the address to the first 13 entries index block.

- 1 Recall
- 2 Introduction
- 3 Allocation approaches
- 4 Free space management
- 5 Repositories
- 6 Partitions
- 7 File system operations**
- 8 Protections

Operations on the *FS* can be grouped in two categories:

- operations on files
- internal operations in files

Operations on files:

- **creating a file:**
 - ▶ *FS* checks if a file with the same name does not already exist in the current repository
 - ▶ If not: *FS* adds entry to repository and allocate blocks in mass storage memory
- **opening a file:**
 - ▶ The *FS* looks for a file with a specific name in the current repository
 - ▶ If user has the rights to open the file (and if it exists): a "**Logical File**" is created (targeting the "**Physical File**")

Operations on files:

- **closing a file:**

- ▶ The *FS* **cut the link between the "Logical File"** (file representation on main memory) and the **"Physical File"** (file representation on mass storage memory)

- **destroying a file:**

- ▶ The *FS* **looks for the file with the corresponding name**
- ▶ If it finds it: the **corresponding entry removed from index and blocks are freed**

Internal operations on files:

- **read/write operations on a file:**

- ▶ The *FS* **keeps track of the next current block position:** to read a new block or to write a new block

- 1 Recall
- 2 Introduction
- 3 Allocation approaches
- 4 Free space management
- 5 Repositories
- 6 Partitions
- 7 File system operations
- 8 Protections**

Why a protection?

The *FS* needs to be protected against two things:

- **Inappropriate file accesses** (no right to open/read or execute a file)
- **Physical damages**

Protection against inappropriate file accesses:

- Files have associated access rights (read, write and execution)
- Files have an associated access list containing:
 - ▶ explicit user names
 - ▶ and/or groups (of user names)

Example after a "ls -l" command :

owner	group	all	owner	size	date	filename
rw-	r- -	r- -	name	size1	date1	file1
rwx	r-x	r-x	name	size2	date2	file2

Protection against physical damages:

Useful?

YES: power cuts, mechanical damages of the disks, dust, high temperatures, etc ...

The *FS* can be protected using two kinds of **redundancy** techniques:

- **Internal redundancy:** two copies of the same data are kept in memory (example: two copies of the FAT) and synchronized at each change
- **Periodic saves redundancy:** a copy is kept on another memory device (external hard drive) and it is synchronized at each change (on *UNIX*: the *rsync* command)